

Projektdokumentation - Kundenauftrag

-Projektarbeit an der BBS1 Mainz-
Schuljahr 2024/2025

Thema:

Teamverwaltungssoftware für ein MOBA-Spiel

Fach: LF8

Abgabedatum: 11.04.2025

Betreuende Lehrkraft: Herr Schumacher

Gruppenmitglieder:

- Louis Lohmer (Anwendungsentwickler)
- Oliver Skronn (Systemintegrator)
- Dominik Krauberger (Systemintegrator)
- Maria Neciporenko (Anwendungsentwickler)

Erklärung über die selbständige Anfertigung

Hiermit erklären wir, dass wir die vorliegende Dokumentation und das Projekt selbstständig angefertigt, keine anderen als die angegebenen Hilfsmittel benutzt und die Stellen der Arbeit und Dokumentation, die im Wortlaut oder im wesentlichen Inhalt aus anderen Werken entnommen wurden, mit genauer Quellenangabe kenntlich gemacht haben.

Ort: Mainz

Datum: 11.04.2025

Inhaltsverzeichnis

AUFGABENSTELLUNG	2
VORGEHENSWEISE UND RELEVANTE KENNDATEN.....	2
ER-MODELL.....	3
DATENBANKSTRUKTUR.....	4
WORKFLOW	4
VERWENDETE SOFTWARE UND TECHNOLOGIEN	5
BERECHNUNG DER ARBEITSSTUNDEN	5

Aufgabenstellung

Ziel der Aufgabe besteht darin, eine lauffähige Teamverwaltungssoftware für MOBA-Spiele im Rahmen des zweiten Lehrjahres an der BBS1 zu entwickeln, welche die folgenden Anforderungen hat:

1. Neue Mitglieder sollen in der angebundenen Datenbank mit Spielernamen, Vornamen, Nachnamen, E-Mailadresse, Spielposition, Elo-Punkte und co. angelegt werden können. Zudem sollen diese auch nachträglich verändert oder gelöscht werden können
2. Es können Teams mit maximal fünf Spielern erstellt, bearbeitet und gelöscht werden
3. 1 vs 1 Matchmaking-Modus, bei dem zwei ausgewählte Spieler jeweils als Sieger und Verlierer entschieden werden, wobei sich das Ergebnis auf die Elo-Punkte auswirkt.
4. Team vs Team Matchmaking-Modus, bei dem zwei ausgewählte Teams jeweils als Sieger und Verlierer entschieden werden, wobei sich das Ergebnis auf die Team Elo-Punkte auswirkt.
5. Turniermodus, bei dem ausgewählte Teams in Runden gegeneinander antreten, bis am Ende ein einzelner Gewinner feststeht.
6. Suchfunktion mit Filteroptionen, beispielsweise für eine spezifische Spielposition

Grundvoraussetzung sind eine Datenbank sowie eine GUI, wobei die Gruppe über die dafür verwendete Technologie entscheiden kann.

Vorgehensweise und relevante Kenndaten

Unser Team besteht insgesamt aus vier Mitglieder, darunter befinden sich zwei Anwendungsentwickler und zwei Systemintegratoren. Zunächst haben wir die Grundlegenden Aufgaben in einer Word-Datei festgehalten, die gleichzeitig als Check-Liste dient.

Allgemeines

Mitglieder: Louis Lohmer (**Anwendungsentwickler**), Oliver Skronn (**Systemintegrator**), Maria Neciporenko (**Anwendungsentwickler**), Dominik Krauberger (**Systemintegrator**)
Aufgabe: [RDBMs v2.3 SchuelerVersion.pdf](#)
Deadline: April 2025

Ziele und To-Dos

Grundlegende Ziele:

- Funktionsfähiges Programm mit allen geforderten Anforderungen:
 - Mitglieder mit Spielernamen, Vor- und Nachname, Spielposition, E-Mail und Elo-Zahl in der Datenbank erstellen, bearbeiten und löschen können (**CRUD-Operation Delete, Create und Update**)
 - Teams mit maximal fünf Mitgliedern erstellen, bearbeiten und löschen können. (**CRUD-Operation Delete, Create und Update**). Mitglieder können Teams nur einem Team angehören
 - Drei verschiedene Modi, unter anderem Matchmaking (1vs. 1 / Team vs. Team) und Turniermodus
 - Suchfunktion nach gewissen Parametern, beispielsweise Spielposition
 - Funktionsfähiger Datenbank | **erreicht**
- Dokumentation:
 - ER-Modell | **erreicht**
 - Kenndaten und Vorgehensweise | **erreicht**
 - Datenbankstruktur | **erreicht**
 - Verwendete Technologie | **erreicht**
 - Monetäre Berechnung der Arbeitszeit
- Rechnung an den Auftragsnehmer (Unsere Stammgruppe)

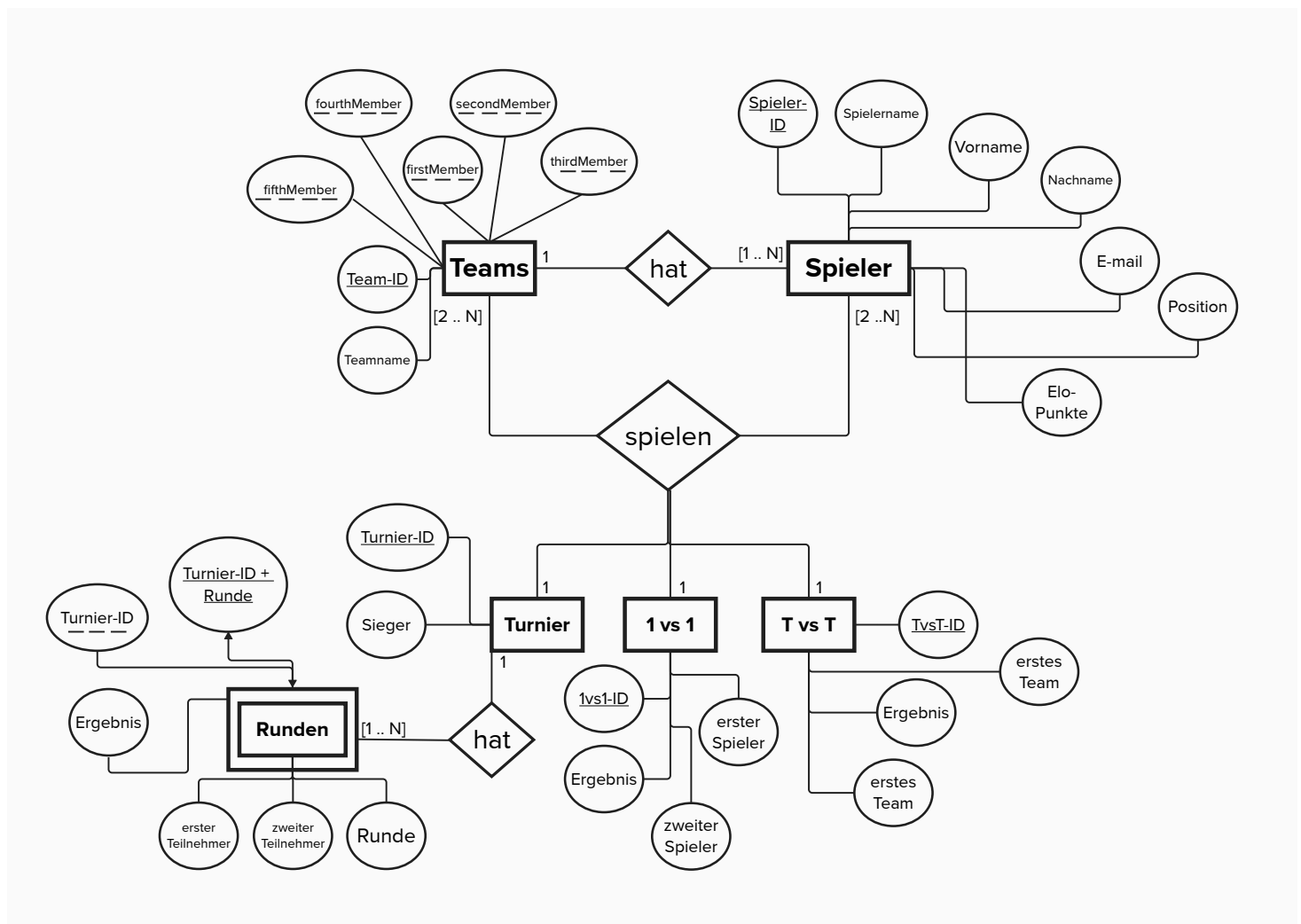
Erweitertes Ziel:

- Neue Feature in die bestehende Software implementieren und diese dokumentieren

Zuerst haben wir die Datenbank erstellt und final zum Laufen gebracht, dabei haben wir die vier notwendigen Schritte beachtet:

1. In der **Informationsanalyse** wurde anhand des Arbeitsauftrages zuerst ermittelt, welche Daten in welcher Form gespeichert werden müssen
2. In der **Konzeptionellen Phase** haben wir in Zusammenarbeit das **ER-Modell** besprochen und erstellt
3. Das **relationale Datenmodell** wurde in der **logischen Phase** zuerst entsprechend der ersten drei Normalformen normalisiert und schlussendlich skizziert
4. In der letzten Phase, die **physische Phase**, wurde die **Datenbank** schlussendlich anhand des relationalen Datenmodells erstellen

ER-Modell



Datenbankstruktur

Tabelle Teams

Team-ID INT	Teamname VARCHAR(255)	Elo-Punkte INT	firstMember INT (Default NULL)	secondMember INT (Default NULL)	thirdMember INT (Default NULL)	fourthMember INT (Default NULL)	fifthMember INT (Default NULL)
1	rot	1200	1	3	5	7	9
2	blau	2300	2	4	6	8	10

Tabelle Spieler

Spieler-ID INT	Spielername VARCHAR(255)	Vorname VARCHAR(255)	Nachname VARCHAR(255)	E-Mail VARCHAR(255)	Position VARCHAR(30)	Elo-Punkte INT	deleted BOOLEAN
1	Max3415	Max	Mustermann	max.mustermann@gmail.com	Top	1200	0
2	LinaB.	Lina	Musterfrau	lina.musterfrau@gmail.com	Top	1200	1

Tabelle T vs T:

TvsT-ID INT	erstes Team VARCHAR(255)	Zweites Team VARCHAR(255)	Ergebnis VARCHAR(20)
1	rot	blau	5 - 0

Tabelle 1 vs 1:

1vs1-ID INT	erster Spieler VARCHAR(255)	Zweiter Spieler VARCHAR(255)	Ergebnis VARCHAR(20)
1	Max3415	LinaB.	4 - 3

Tabelle Turnier:

Turnier-ID INT	Sieger VARCHAR(127)
1	rot

Tabelle Runden:

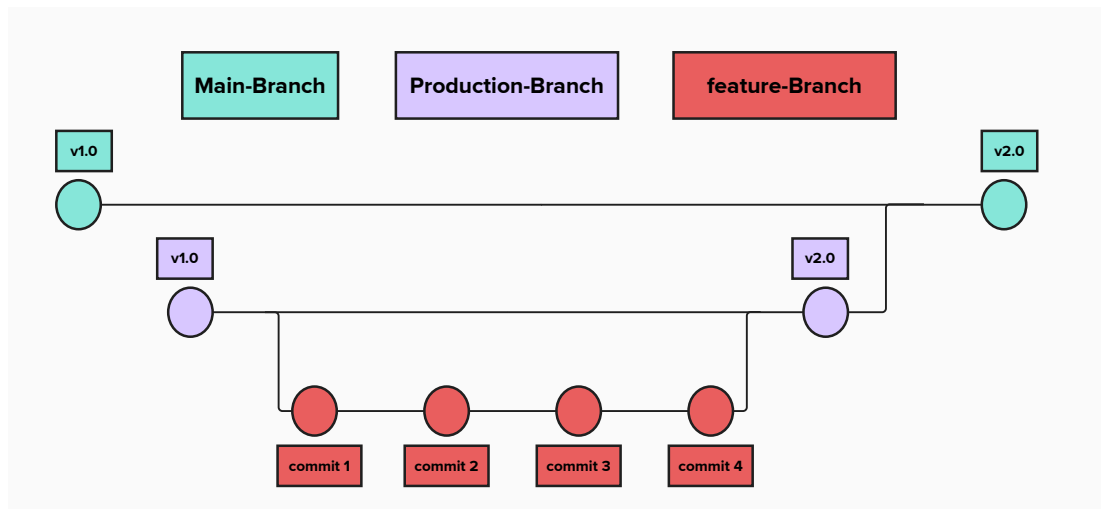
Runden-ID INT	Turnier-ID INT	Runde INT	erster Teilnehmer VARCHAR(255)	zweiter Teilnehmer VARCHAR(255)	Ergebnis VARCHAR(20)
11	1	1	rot	blau	5 - 0
12	1	2	blau	rot	3 - 4

Workflow

Darauffolgend begann die Entwicklung nach dem top-down Prinzip. Die Übersichtsseiten wurden erstellt, worauf die einzelnen Unterkategorien und letztlich Inhalte folgten. Wir haben uns für einen feature-based workflow entschieden, sodass wir kurzlebige Branches für die Erfüllung der Aufgaben hatten, während wir einen Production Branch zur Absicherung und den Main Branch als Hauptbranch nutzten. Wenn eine neue Komponente oder ein neues Feature in das Projekt implementiert werden soll, wurden zuerst alle Änderungen vom Main-Branch mit dem Befehl „git pull“ lokal geholt und schlussendlich ein neuer Branch, auf Basis des Main-Branch, für das eigentliche Feature erstellt. Sinn und Zweck des Production-Banches ist es, die Integrität des Main-Banches zu schützen und die Software zu mit allen neuen Änderungen testen zu können.

Nachdem die Anforderungen erfüllt waren, wurden weitere mögliche Fehlerquellen durch ausgiebiges Testen beseitigt und das Nutzererlebnis durch das responsive Styling aufgewertet.

Wir haben Verbesserungsvorschläge und Bugs stets festgehalten, worauf sich die Gruppenmitglieder diese dann frei, unter anderem über die Issues-Funktion auf Github, aussuchen konnten.



Verwendete Software und Technologien

Für das Entwickeln der Teamverwaltungssoftware wurde das JavaScript-Framework [Vue.js](#) verwendet, welches mit dem Web-Stack, sprich HTML, CSS und JavaScript, arbeitet. Grund dafür ist, dass Frameworks die Softwareentwicklung bei komplexeren Projekten beispielsweise über eingebaute Funktionen und Unit-Tests vereinfachen. Vue.js bietet zudem den Vorteil, vergleichsweise einfach und intuitiv erlernbar zu sein, auch wenn man bisher noch nicht so viel Erfahrung mit Frameworks hatte.

Die Datenbank wurde mit MySQL-Queries mithilfe von phpMyAdmin erstellt und verwaltet, da dieses im Vergleich zu den verschiedenen Datenbankverwaltungssystemen von SQL sehr stark verbreitet ist. Abgesehen davon bietet es Zuverlässigkeit und Schnelligkeit, was sich zudem positiv auf die Performance der Anwendung bei Datenbankabfragen auswirkt.

Damit Frontend und die Datenbank miteinander kommunizieren können, braucht es Node.js und das Framework Express.js. Mithilfe von Node.js kann über eine REST-API mit Endpoints eine Verbindung zur Datenbank hergestellt werden. Allerdings läuft Node.js nur auf dem Client-Gerät und kann nicht direkt mit der Anwendung, die auf einem Server läuft, kommunizieren. Daher wird mit Express.js ein http-Server bereitgestellt, sodass man von der Anwendung direkte http-Anfragen an die Datenbank stellen kann.

Schlussendlich wurde die Container-Software Docker verwendet, um das Frontend und das Backend zu containerisieren. Der Vorteil für die Entwickler ist dabei, dass die gesamte Entwicklung vereinfacht wird. Beispielsweise muss die Datenbank nach neuen Änderungen nicht mehr manuell geteilt werden. Der Nutzer hingegen hat den Vorteil, dass die Anwendung nutzerfreundlicher ist, weil alle notwendigen Abhängigkeiten schon installiert sind.

Berechnung der Arbeitsstunden

Nachfolgend befindet sich die Rechnung des Projektes:

Smart IT GmbH – Besserwisserbach, 666 – 12345, Nichtzustimmungen
Joshua Schumacher
Am Prügelweg 3
55118 Testosteronheim



Besserwisserbach, 666
12345, Nichtzustimmungen

E-Mail: supportSmartIT@gmail.com

Steuernummer: DE496210657

Telefon: +49 491/873838976

Vat-Nummer: DE496210657

EORI-Nummer: DE954413948633809

WEEE-Nummer: 52527346

Datum: 11.04.2025

Kunden-Nr.: 28323785

Rechnungs-Nr.:2018967

Rechnung

Sehr geehrter Herr Schumacher,

vielen Dank für Ihre Bestellung. Anbei erhalten Sie ihre Rechnung ihrer Bestellung vom 11.04.2025 mit der Bestellungsnummer 2018967. Die Rechnung wurde bereits am 11.04.2025 über PayPal bezahlt.

LEISTUNG	STUNDEN	PREIS/STUNDE	PREIS GESAMT
KICK-OFF MEETING UND WEITERE BESPRECHUNGEN	4	45€	180€
ER-MODELL ERSTELLEN	3	45€	135€
ERSTELLEN UND EINRICHTEN DER DATENBANK	5	45€	225€
CONTAINERISIERUNG	2	45€	90€
DOKUMENTATION SCHREIBEN	3	45€	135€
PROJEKT VOLLSTÄNDIG FUNKTIONAL PROGRAMMIEREN	35	45€	1575€
PROJEKT RESPONSIV DESIGNEN	7	45€	315€
TESTEN UND BUGS BEHEBEN	5	45€	225€

Summe Netto: 2.880€
MwSt. 19% : 547.20€
Gesamtbetrag: 3427.20€

Zahlungsempfänger: Smart IT GmbH
Bankdaten: Volksbank Raifeisenbanken AG
BIC: HVYEDENN477, IBAN: DE53100408900033718751

